

Topics in Quantitative Finance - Homework 1

Assigned: July 24, 2026. Due: **Monday, July 27, 2026** by 1PM.

Late homework **will not be accepted**.

Homework is to be done by each student individually. To receive full credit, you must email a completed copy of this Jupyter notebook to TAs at topics_in_qf@163.com by the due date and time. All codes must run correctly and solutions must be written up neatly in Markdown/LaTeX format. If you encounter problems with Jupyter notebook, please contact TA 李新宇 or 林文鑫.

Name: msj

B_t denotes a standard Brownian motion in the following.

1. (5 points)

Let $X_t = kB_{t/k^2}$, where $k \neq 0$ is constant. Show by definition that X_t is also a standard Brownian motion.

Solution 1.

We only need to verify the followings:

1. Initial value is 0: $X_0 = kB_0 = 0$ a.s. .

2. Gaussian increments: for $t, s \geq 0$,

$$X_{t+s} - X_s = k(B_{t+s/k^2} - B_{s/k^2}).$$

Since $B_{t+s/k^2} - B_{s/k^2} \sim N(0, \frac{t}{k^2})$,

$$X_{t+s} - X_s \sim N(0, t).$$

3. Independent increments: For $0 \leq t_0 \leq t_1 \leq \dots \leq t_m$, since $k^2 > 0$, we still have

$$0 \leq \frac{t_0}{k^2} \leq \frac{t_1}{k^2} \leq \dots \leq \frac{t_m}{k^2}.$$

The time points are increasing. Hence the increments

$$B_{t_j/k^2} - B_{t_{j-1}/k^2}, \quad j = 1, \dots, m,$$

are independent. Multiplying by the constant $\sqrt{k}$ preserves independence, so the increments of $\tilde{X}_t$ are independent.

4. Continuity: For almost every sample point ω , the map $u \mapsto B_u(\omega)$ is continuous. Since $t \mapsto t/k^2$ is continuous, $t \mapsto \tilde{X}_t(\omega) = \sqrt{k} B_{t/k^2}(\omega)$ is also continuous.

Hence we are done!

Donsker's invariance principle

Recall the Donsker's invariance principle:

Suppose $\{X_i\}_{i=1}^\infty$ is an iid sequence of random variables with mean 0 and variance 1. Let $S_n = \sum_{i=1}^n X_i$. Define the function $\mathfrak{S}_n$ of t by

$$\mathfrak{S}_n(t) = \frac{1}{\sqrt{n}} \sum_{i=1}^n \left[S_{i-1} + n \left(t - \frac{i-1}{n} \right) X_i \right] \mathbf{1}_{\left(\frac{i-1}{n}, \frac{i}{n}\right]}(t).$$

In fact, $\mathfrak{S}_n$ is simply the linear interpolation of the scaled random walk $\left\{ \frac{S_1}{\sqrt{n}}, \frac{S_2}{\sqrt{n}}, \dots, \frac{S_n}{\sqrt{n}} \right\}$.

Then, $\mathfrak{S}_n \Rightarrow W$ as $n \rightarrow \infty$, where W denotes a standard Brownian motion.

In other words, as $n \rightarrow \infty$, the linearly interpolated scaled random walk $\mathfrak{S}_n$ converges weakly to a standard Brownian motion.

2. (10pts)

In this problem, we show, numerically by simulation, the Donsker's invariance principle with the iid sequence $\tilde{X}_t$ being

- (a) $\tilde{X}_t \sim \chi^2_3$, i.e., the χ^2 distribution with degrees of freedom 3;
- (b) $\tilde{X}_t \sim \exp(1)$, i.e., exponential distribution with mean 1.

Consider the Brownian motion in the time interval $[0, 2]$, use 100 time steps and generate $15,000$ sample paths to reproduce the plots for sample mean, sample standard deviation, and the histograms at $t = 1$ and $t = 2$. To begin with, please set the random seed by including `numpy.random.seed(9527)` on top of your code.

```
In [1]: # as always, import required modules and functions
import numpy as np
from numpy import sqrt, log, exp
import matplotlib.pyplot as plt
import scipy.stats as ss
from scipy.stats import norm
import seaborn as sns
```

Solution 2(a).

For t distribution, $E(T) = 0$, $Var(T) = \frac{3}{3-2} = 3$

Donsker's Thm requires iid and variance one, so we use $\frac{x_t - \bar{x}_t}{\sqrt{3}}$

```
In [2]: # insert python code for (2a) in this cell
np.random.seed(9527)

# Simulate BM by using the Donsker's invariance principle
# number of samples, number of steps, terminal time
n_sim, n_steps, T = 15000, 100, 2
dt = T/n_steps

# initialize the Brownians
B = np.zeros([n_sim, n_steps+1])

# simulation step
for i in range(n_steps):
    dB = ss.t.rvs(df=3, size=n_sim)
    # dB = dB - dB.mean() # now dB has mean 0
    # dB = dB/dB.std() # now dB has variance 1
    # t_3 mean 0 var=3/(3-2)=3
    dB = dB/sqrt(3)
    dB = np.sqrt(dt)*dB
    B[:,i+1] = B[:,i] + dB

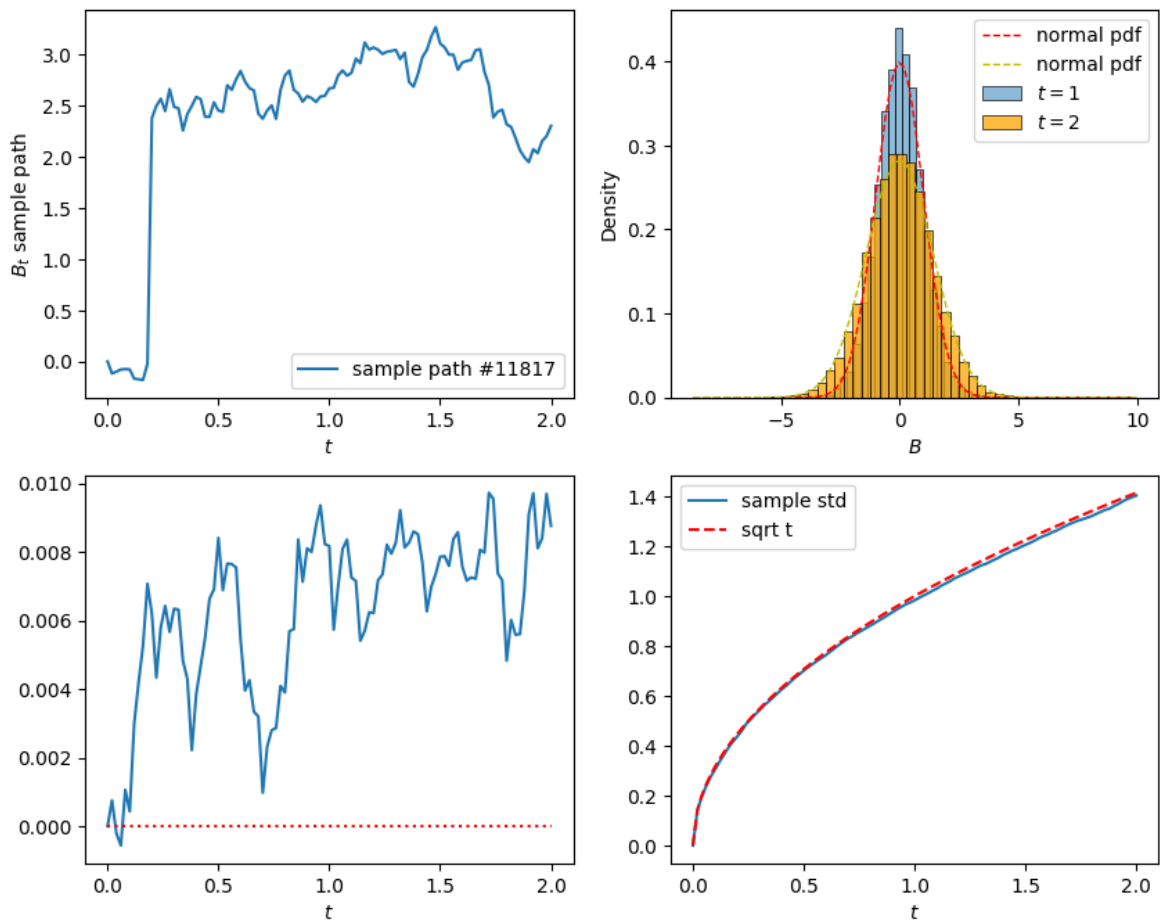
#=====
# plot
t = np.arange(0, T+dt, dt)
#t = np.linspace(0, T, n_steps+1)
plt.figure(figsize=(10, 8))

# sample path
path = np.random.choice(n_sim)
plt.subplot(2, 2, 1)
plt.plot(t, B[path, :], label=f'sample path #{path}')
plt.xlabel('$t$')
plt.ylabel('$B_t$ sample path')
plt.legend()

# histograms
plt.subplot(2, 2, 2)
time = 1
sns.histplot(B[:,t==time], bins=50, stat='density', label=f'$t={time}$')
sns.histplot(B[:, -1], bins=50, stat='density', color='orange', label=f'$t={T}$')
x = np.linspace(B[:, -1].min(), B[:, -1].max(), 100)
plt.plot(x, norm.pdf(x), 'r--', lw=1, label='normal pdf')
plt.plot(x, norm.pdf(x, scale=sqrt(2)), 'y--', lw=1, label='normal pdf')
plt.xlabel('$B$')
plt.legend();

# sample mean
plt.subplot(2, 2, 3)
plt.plot(t, B.mean(axis=0))
plt.hlines(y=0, xmin=0, xmax=T, color='red', ls='dotted')
plt.xlabel('$t$')
```

```
# sampel standard deviation
plt.subplot(2, 2, 4)
plt.plot(t, B.std(axis=0), label='sample std')
plt.plot(t, np.sqrt(t), 'r--', label='sqrt t')
plt.xlabel('$t$')
plt.legend();
```



Solution 2(b).

similar to 2(a), we use $X_i = T_i - 1$

```
In [3]: # insert python code for (2b) in this cell
np.random.seed(9527)

# Simulate BM by using the Donsker's invariance principle
# number of samples, number of steps, terminal time
n_sim, n_steps, T = 15000, 100, 2
dt = T/n_steps

# initialize the Brownians
B = np.zeros([n_sim, n_steps+1])

# simulation step
for i in range(n_steps):
    dB = ss.expon.rvs(size=n_sim)
    # dB = dB - dB.mean() # now dB has mean 0
    # dB = dB/dB.std() # now dB has variance 1
    # exp(1) mean 1 var 1
    dB=dB-1
```

```

dB = np.sqrt(dt)*dB
B[:,i+1] = B[:,i] + dB

#=====
# plot
t = np.arange(0, T+dt, dt)
#t = np.linspace(0, T, n_steps+1)
plt.figure(figsize=(10, 8))

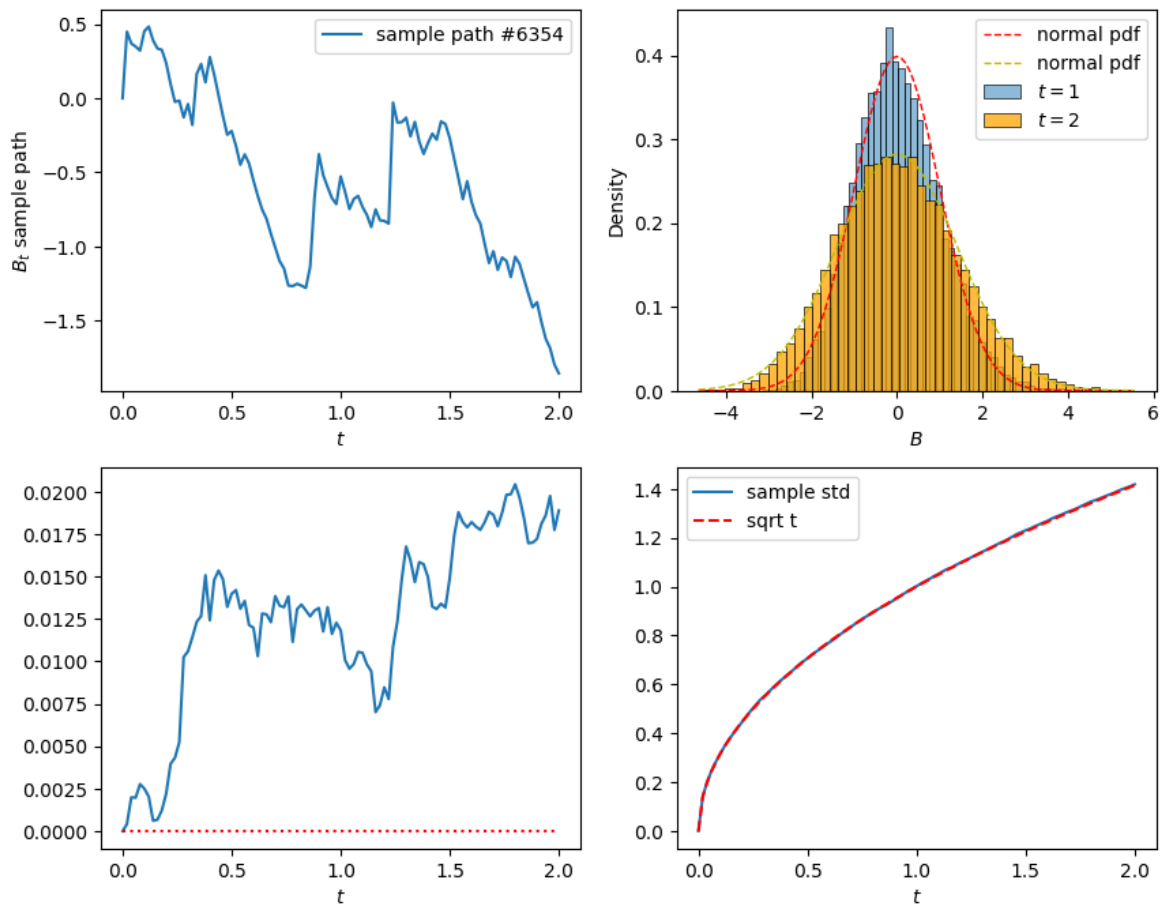
# sample path
path = np.random.choice(n_sim)
plt.subplot(2, 2, 1)
plt.plot(t, B[path, :], label=f'sample path #{path}')
plt.xlabel('$t$')
plt.ylabel('$B_t$ sample path')
plt.legend()

# histograms
plt.subplot(2, 2, 2)
time = 1
sns.histplot(B[:,t==time], bins=50, stat='density', label=f'$t={time}$')
sns.histplot(B[:, -1], bins=50, stat='density', color='orange', label=f'$t={T}$')
x = np.linspace(B[:, -1].min(), B[:, -1].max(), 100)
plt.plot(x, norm.pdf(x), 'r--', lw=1, label='normal pdf')
plt.plot(x, norm.pdf(x, scale=sqrt(2)), 'y--', lw=1, label='normal pdf')
plt.xlabel('$B$')
plt.legend();

# sample mean
plt.subplot(2, 2, 3)
plt.plot(t, B.mean(axis=0))
plt.hlines(y=0, xmin=0, xmax=T, color='red', ls='dotted')
plt.xlabel('$t$')

# sampel standard deviation
plt.subplot(2, 2, 4)
plt.plot(t, B.std(axis=0), label='sample std')
plt.plot(t, np.sqrt(t), 'r--', label='sqrt t')
plt.xlabel('$t$')
plt.legend();

```



3. (10pts)

Alter the python code provided in the lecture to simulate the process X_t given by

$$X_t = 1 + 3t(2 - t) + B_{t_s}$$

for $0 \leq t \leq 2$. Use 200 time steps and generate 20,000 sample paths, plot sample mean, sample standard deviation (as functions of t), and the histograms at $t = 1$ and $t = 2$. To begin with, set `numpy.random.seed(1732)`.

Solution 3.

```
In [4]: # insert python code for (3) in this cell
np.random.seed(1732)

# Simulate BM by using the Donsker's invariance principle
# number of samples, number of steps, terminal time
n_sim, n_steps, T = 20000, 200, 2
dt = T/n_steps

# initialize the Brownians
B = np.zeros([n_sim, n_steps+1])

# simulation step
for i in range(n_steps):
    dB = norm.rvs(size=n_sim)
    dB = np.sqrt(dt)*dB
```

```

    B[:,i+1] = B[:,i] + dB

t = np.linspace(0, T, n_steps+1)

# X_t 仍然用变量B存储
B = B + 1 + 3*t*(2-t)

#=====
# plot
t = np.arange(0, T+dt, dt)
#t = np.linspace(0, T, n_steps+1)
plt.figure(figsize=(10, 8))

# sample path
path = np.random.choice(n_sim)
plt.subplot(2, 2, 1)
plt.plot(t, B[path, :], label=f'sample path #{path}')
plt.xlabel('$t$')
plt.ylabel('$B_t$ sample path')
plt.legend()

# histograms
plt.subplot(2, 2, 2)
time = 1
sns.histplot(B[:,t==time], bins=50, stat='density', label=f'$t={time}$')
sns.histplot(B[:, -1], bins=50, stat='density', color='orange', label=f'$t={T}$')

#x = np.linspace(B[:, -1].min(), B[:, -1].max(), 100)
#plt.plot(x, norm.pdf(x), 'r--', lw=1, label='normal pdf')
#plt.plot(x, norm.pdf(x, scale=sqrt(time)), 'y--', lw=1, label='normal pdf')
x = np.linspace(B.min(), B.max(), 300)
#  $X_1 \sim N(4, 1)$ 
plt.plot(x, norm.pdf(x, loc=4),
         'r--', lw=1, label='N(4,1)')
#  $X_2 \sim N(1, 2)$ 
plt.plot(x, norm.pdf(x, loc=1, scale=sqrt(2)),
         'y--', lw=1, label='N(1,2)')

plt.xlabel('$B$')
plt.legend();

# sample mean
plt.subplot(2, 2, 3)
plt.plot(t, B.mean(axis=0))
# plt.hlines(y=0, xmin=0, xmax=T, color='red', ls='dotted')
plt.plot(t, 1 + 3*t*(2-t), 'r--', label='1+3t(2-t)')
plt.xlabel('$t$')
plt.legend()

# sampel standard deviation
plt.subplot(2, 2, 4)
plt.plot(t, B.std(axis=0), label='sample std')
plt.plot(t, np.sqrt(t), 'r--', label='sqrt t')
plt.xlabel('$t$')
plt.legend();

```

